L Systems

Implementazione in raylib e c++

B. Rodolfo, H.Kirollos

https://github.com/alonzo-bazaar/lsystems

Contents

L Systems

General Anatomy of an L-System

To draw a tree using L-systems we need to have all components of an L-system first, these are

- An alphabet of **symbols** which will be used as instructions for drawing the tree these symbols may be
  - **parameterized**: an instruction character accompanied by a vector of parameters (a vector $\in \mathbb{R}^n$ for some $n \in \mathbb{N}^*$)
  - **non parameterized**: an instruction character with no accompanying parameters
- **Rewrite rules** which will be used to iteratively process a string made out of these symbols, these rewrite rules may be
  - **deterministic** or **stochastic** (a stochastic rule applies a rule sampled from a distribution of deterministic rules)
  - **context free** (known as **0L-systems**) or **context dependant** (such as **2L-systems**)
- A **turtle graphics** system, which will take the processed symbol string and use it as instructions to go and draw the tree

Our L-System

For this lab project we have implemented a **stochastic, parametric, bracketed 0L-System**

- **0L-System**: means the rewrite rules (sometimes called **production rules**) are **context free**, this means that when determining how to replace a symbol, only that symbol is considered, and its context (eventual nearby symbols) is not taken into consideration
- **Stochastic**: the production rule we apply when rewriting a symbol may be chosen in a deterministic manner or sampled from a distribution of predefined productions
- **Parametric**: The strings our turtle and productions act on are composed of symbols which may include an accompanying vector of parameter
- **Bracketed**: The system has a state stack, which allows for greater expressivity when defining branching structures

Turtle Graphics

Traditional Turtle Graphics Systems

Discussion of our symbol alphabet, and, by extension, of the instruction strings we act on, are best understood with the context of the system these aphorementioned instructions act upon.

Traditionally, a **turtle graphics** system is comprised of

- A turtle **position** ($\in \mathbb{R}^2$) and an orientation/angle the turtle is at
- A "**pen**", which may be **up** or **down** to draw or not draw a line where the turtle moves

Such a system is controlled by a rather reduced instruction set, usually

- Go **forward** for a given length
- Turn **left** or **right** by a given angle
- **Raise** pen if pen is not up, or **lower** pen if pen is not down

Instructions for our Turtle Graphics System

The turtle graphics system presented in the book differs from traditional turtle graphics in a number of ways

- It acts in **3D space**, allowing for rotations of **pitch**, **yaw**, and **roll**, instead of simply left or right
- It may **save** a previous state, and **restore** a previously saved state
- It has a **thickness** and a **color** table to determine how the “pen” draws, indices in these tables are part of the state and may be incremented or decremented
- It does not have a notion of pen up/down, it instead encodes “move leaving a mark” and “move without leaving a mark” as **two different instructions**
- It may enter or leave a so called “**polygon mode**”, where
  - while in polygon mode all positions at which the turtle stops after an instruction are saved in an internal buffer
  - upon exiting polygon mode a polygon is between all these positions

Structure of our Turtle Graphics System

These are the instructions the turtle needed to implement as per the book, to create a system capible of executing these instructions our turtle includes

- A **position** in $\mathbb{R}^3$, and an **orientation** represented using **3 unit vectors**, describing the H (heading), L (left), and U up, directions of the turtle, arranged as columns of a HLU matrix
- "**color**" and **thickness** tables, passed at construction time, and **indices** within those table, as state variables
  - to better work with our shader we have opted to use a **texcoords table** instead of a color table, color table behaviour is **emulated** using increasing texcoords and a vertical gradient texture
- a **stack** of previously saved states, a **frame** within this stack includes position, orientation, and indices within the color and thickess tables

Structure of our Turtle Graphics System

Furthermore, to implement the **polygon mode** described in the book, the turtle includes

- A **flag** controlling wether we are in polygon mode or not
- A resizeable buffer where all visited positions are saved whilst moving around in polygon mode

As a performance consideration, instead of **drawing** the tree at **each frame**, the turtle graphics system is instead tasked with **creating the tree mesh** only **once**.
To produce this tree mesh the turtle includes

- a resizeable buffer of **vertex coords**
- a resizeable buffer of **texture coords**

to which it adds all vertices (branches or leaves) it is tasked with drawing, and which gets turned into a mesh once the mesh creation is finished, following a **builder/director design pattern**.

Symbols and Rewrite Rules

Symbols

Symbols in an instruction string may be either special symbols accepted by the turtle, or intermediate symbols, used by the rewrite step, but ignored by the turtle. The symbols accepted by the turtle are

- `F` or `f`, with one parameter, indicating a **stride**
  intepreted as: go forward by a length `stride`, leaving (`F`) or not leaving (`f`) a mark.
- `&`, `^`, `+`, `-`, `/`, or `\`, with one parameter indicating **angle**
  interpreted as: rotate by an angle `angle`, clockwise (`^`, `-`, `/`) or counter clockwise (`&`, `+`, `\`), around the H (`/`, `\`), L (`&`, `^`), or U (`+`, `-`) axis
- `!`, or `'`, with no parameters
  interpreted as: increment the index in the thickess (`!`) or color (`'`) table
- `[`, or `]`, with no parameters
  interpreted as: push state to stack (`[`) or restore last state from stack (`]`)
- `{`, or `}`, with no parameters
  interpreted as: enter (`{`) or leave (`}`) polygon mode

Rewrite Rules

The rewriting engine follows the following logic

- A rewritten string is obtained by concatenating the rewriting of all its symbols (flatmap), symbols for which no matching rewrite is found are rewritten into themselves
- A production is an aribrary function of a symbol's parameter vector, the production a symbol is passed to is determined only by the letter/instruction in the symbol

Stochastic and Deterministic Rewrite Rules

As stated above our rewrite system supports both deterministic and stochastic productions

- A deterministic production is a (deterministic) function of the symbol's vector parameters
- A stochastic production (by the book) is a finite set of deterministic productions, each with a probability of being chosen, a stochastic production is applied by sampling a deterministic production from this set and applying it

Terrain

Terrain

- Procedural terrain generation
- Based on Fractal Perlin Noise chunks
  - using non precalculated vectors
- Amplitude, frequency, lacunarity are some of the attribute that models the shape

Terrain Management

- A function in the Terrain class take care of the chunks generation
  - Using a value of Render Distance
    - When the player move and a rendered chunk became outside the RD
    - The Management Function delete the chunk and deallocate memory
- Every chunk generated comes with a mesh and a bounding box
  - the bbox is used for Frustum Culling

Terrain Drawing

- Only the chunks inside the Render Distance are drawn
  - Since they are the only still existing
- Applied Frustum Culling
- Terrain texture has diffrent textures depending on the height

Texture e Shader

Texturing

- Readapted a PBR shaders to use textures and lighting
- An MRA image is created using Roughness and Ambient Occlusion
  - From the loaded textures
  - Metalness is set to 0
- Then is used by the shaders togheter with a una Normal map

Shader

- PBR shader used for lightning interaction with textures
- A shader is used for the trees another for the terrain
  - Separation is needed since terrain use two textures
- Light class has been made for create directional light
  - Rapresenting the sun
  - Can manage more light by the shaders

Player

Player

- Class that manage the camera and player controls
- Use a semi-realistic phisics with differen parameter s.a.
  - Gravity
  - Air drag
  - Ground drag
  - Bobbing
- Has the attribute of the Frustum Planes
  - Needed for frustum culling